ONLINE QUIZ AND QUESTION BANK MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

REGNO	NAME
111624102217	**TAMIL MANI B**
111624102235	**VARUNESH D**
111624102242	**VISHWAJITH R**
111624102225	**THEJA J G**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

PUDUVOYAL



OCTOBER 2026

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
R.S.M. Nagar, PUDUVOYAL-601 206
Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

BONAFIDE CERTIFICATE

This is to certify that **TAMIL MANI B, VARUNESH D, VISHWAJITH R and THEJA J G**, bearing **Register Numbers 111624102217, 111624102235, 111624102242 and 111624102225** respectively, of **3rd Year, 5th Semester, Department of Computer Science and Engineering**, have successfully completed the mini project entitled **"Online Quiz and Question Bank Management System"** during the academic year **2026–2027** under my guidance and supervision.

The work submitted in this report is a bonafide record of the work carried out by the student/team as part of the academic requirements of the course.

Name of the Student	**Project Guide**
TAMIL MANI B	Mrs. R. Saranya
VARUNESH D	
VISHWAJITH R	
THEJA J G	

ABSTRACT

Online quizzes are usually little more than a list of questions and a score. Teachers get no help in keeping a good question bank, students get no advice on what to study next, and timing and cheating are easy to abuse. This mini project builds the back end of an Online Quiz and Question Bank Management System as a REST API with 29 operations on 24 resources, written in Python with FastAPI and Pydantic, with SQLite on a laptop and PostgreSQL when deployed. Three roles (admin, teacher, student) use bearer-token authentication, and passwords are stored with salted PBKDF2. Teachers keep a question bank with filters, bulk import from CSV and an explanation for every answer, and build quizzes by hand or at random. A quiz attempt is controlled by the server: the timer, the number of attempts, a per-student shuffle of questions and options, and the exclusion of late attempts from the leaderboard. Beyond marking, the system asks each student how sure they are of every answer and computes a calibration score from it (100 minus the Brier score); wrong answers given with certainty are flagged as likely misconceptions and, like every wrong answer, return for revision after 1, 2, 4, 8 and 16 days (Leitner boxes). An adaptive practice mode chooses the weakest topic and a difficulty that follows the last six answers, and teachers get item analysis (difficulty and discrimination index) that shows which questions are mislabelled or do not separate strong from weak students. The system is checked by 13 automated tests. In a simulation of 3,000 attempts for each of four kinds of student, the calibration score separated an honest student (80) from an always-certain one (64) although both answered 60% of the questions correctly; adaptive practice gave a strong student 77% hard questions and a weak student only 8%.

TABLE OF CONTENTS

S.No	Section	Page No.
1	Introduction	1
2	Objectives	4
3	Tools and Technologies	6
4	Architecture Diagram	7
5	Methodology	10
6	Results	15
7	Conclusion	29
8	Reference	31

1. INTRODUCTION

1.1 Background

Colleges and training centres use online quizzes for revision, for weekly tests and for placement practice. A quiz system has to store questions, assemble them into quizzes, let students attempt the quizzes under a time limit, mark the answers and show the results to the teacher. Most small systems stop there. They do not tell the teacher which questions are badly written, they do not tell the student what to revise, and they leave the timer, the shuffling and the number of attempts to the browser, where a student can change them.

The programming interface of such a system is naturally a **REST API**. REST (Representational State Transfer) is the architectural style described by Fielding [1] in which every thing the system manages, such as a question, a quiz or an attempt, is a *resource* with its own address, and the standard HTTP methods are used on it: GET to read, POST to create, PUT to replace and DELETE to remove [2]. The server keeps no session between requests; each request carries what the server needs, for example a bearer token [3]. The answer is a JSON document and a status code that says what happened: 200 for success, 201 for a created resource, 401 when the caller is not identified, 403 when the caller is not allowed, 404 when the resource does not exist, 409 for a conflict and 422 for invalid input. Because the interface is only a set of addresses and JSON documents, the same API can serve a web page, a mobile app, a test script or a teacher's spreadsheet.

Table 1.1: REST principles and how this project applies them

Principle	Meaning	In this project
Resources	everything has an address	/api/questions, /api/quizzes/{id}, /api/attempts/{id}
Uniform interface	standard methods with standard meanings	GET reads, POST creates, PUT replaces, DELETE removes
Stateless	no session kept on the server	every request carries 'Authorization: Bearer <token>'
Representations	resources are exchanged as documents	JSON in and out; CSV export of results
Status codes	the outcome is part of the response	200, 201, 204, 400, 401, 403, 404, 409, 422 used as in Section 5.2
Self-description	clients can discover the interface	OpenAPI document and Swagger UI generated at /docs

1.2 Problem Statement

A quiz system that only marks answers has four weaknesses. First, the question bank grows without feedback: nobody finds out that a question labelled easy is answered

wrongly by most students, or that a question is answered correctly by strong and weak students alike. Second, a student who has finished a quiz learns a score, not what to do next; a wrong answer is forgotten after the page is closed. Third, the system cannot tell a student who guessed and was right from one who knew the answer, nor a student who was *sure* and wrong, whose misconception is the most important thing to correct. Fourth, if the timer and the paper are controlled by the browser, the time limit and the order of questions are only suggestions.

The problem addressed here is to design and build the REST API of a quiz and question-bank system that (i) manages users with roles, a question bank and quizzes; (ii) conducts attempts under rules enforced by the server; (iii) goes beyond the score by measuring how well a student's confidence matches the result, by bringing wrong answers back at growing intervals and by choosing practice questions adaptively; (iv) gives teachers measurements of the quality of every question; and (v) is verified by automated tests and by experiments, and can be run on a laptop and deployed with a different database.

1.3 Motivation

The motivation is the gap between what the learning research recommends and what ordinary quiz software does. Reviews of study techniques find that practice testing and spreading the practice over time are among the most effective ways to learn [13][7], yet a typical quiz tool shows each question once. The Leitner system [6] is a simple way to spread revision, the Brier score [5] is a standard measure of how good probability statements are, and classical item analysis [8][9] is a standard way to judge a test question. None of these ideas is new, but they are rarely found together in a student project, and they fit naturally into a REST API in which every piece is a resource that can be tested separately.

1.4 Scope

The deliverable is the **API** and a plain HTML and JavaScript page that uses it, so that the whole system can be demonstrated. The API covers accounts and roles, the question bank (with filters, paging and CSV import), quizzes (manual and random), attempts with a server timer, results, CSV export, item analysis and leaderboard for teachers, and adaptive practice, spaced revision and a mastery map for students. Questions are single-answer multiple-choice. The project does not include e-mail verification, password reset, proctoring, essay questions or a mobile client. The experiments of Chapter 6 use simulated

students and a seeded demonstration database, not real students; the limits are listed in Section 7.1.

2. OBJECTIVES

The main objective is to build a REST API for an online quiz and question-bank system whose behaviour is enforced by the server, and to add learning features that go beyond marking. The specific objectives are:

- To design a resource-oriented API with correct HTTP methods and status codes, documented automatically in OpenAPI and Swagger UI.
- To authenticate users with bearer tokens, store passwords safely, and restrict every operation to the roles that need it.
- To validate every input with declared rules, so that wrong data is rejected with 422 before any logic runs.
- To manage a question bank with filters, paging, bulk CSV import and explanations, and to build quizzes by hand or at random.
- To conduct attempts under server-side rules: timer with a grace period, limited attempts, per-student shuffling and exclusion of late attempts from rankings.
- To compute a confidence-calibration score and to flag confident wrong answers; to schedule revision with Leitner boxes; and to choose practice questions adaptively.
- To give teachers item analysis, results, CSV export and a leaderboard.
- To verify the system with automated tests and with experiments, and to keep it portable between SQLite and PostgreSQL.

Table 2.1: Functional requirements

ID	Requirement
FR1	Register and log in; three roles: admin, teacher, student; a user cannot register as admin
FR2	Teachers and admins create, read, update and delete questions; list them with subject, topic, difficulty and text filters and paging
FR3	Import many questions from CSV and report the lines that are wrong; refuse to delete a question used in a quiz
FR4	Create a quiz from chosen questions or from random questions matching subject, topic and level
FR5	A student starts an attempt (limited number per quiz) and receives a shuffled paper and the time left; a student submits answers with a confidence level
FR6	The server marks the attempt, marks late submissions, computes the calibration score and returns a review with explanations
FR7	Wrong answers enter a revision queue with growing intervals; due questions are served to the student
FR8	Adaptive practice: next question by weakest topic and recent performance; mastery map by topic
FR9	Teachers see results, hardest questions, item analysis, leaderboard and a CSV export; admins manage users and see statistics

Table 2.2: Non-functional requirements

Requirement	How it is met
Security	salted PBKDF2 password hashes; random bearer tokens; role check on every protected route; parameterised SQL; correct answers never sent to a student before submission
Correctness	13 automated tests of the whole API; experiments with simulated students
Fairness	timer and attempts enforced on the server; shuffling derived from the attempt; late attempts excluded from rankings
Portability	SQLite locally, PostgreSQL when DATABASE_URL is set; the SQL is written once
Usability of the API	automatic documentation at /docs; clear error messages in every response
Speed	typical requests take about 5 ms on a laptop (Section 6.10)

3. TOOLS AND TECHNOLOGIES

The back end uses a small set of widely used Python libraries; the front end uses none. Table 3.1 lists what was used and why.

Table 3.1: Tools and technologies used

Item	Tool	Purpose
Language	Python 3.10 or newer (tested on 3.13)	All server code
Web framework	FastAPI 0.115	Routing, dependency injection for authentication and database, automatic OpenAPI documentation
Validation	Pydantic (with FastAPI)	Declares the shape of every request and rejects bad input with 422
Server	Uvicorn	Runs the FastAPI application
Database	SQLite (standard library) and PostgreSQL through psycopg	Storage; one wrapper hides the difference
Security	`hashlib.pbkdf2_hmac`, `secrets`, `hmac`	Password hashing, token generation, constant-time comparison
Front end	HTML, CSS and JavaScript (`fetch`)	A page that uses the API; no framework
Testing	pytest and FastAPI TestClient (httpx)	13 tests that call the API as a client would
Documentation	Swagger UI at `/docs`, OpenAPI at `/openapi.json`	Generated from the code
Deployment	Vercel and Neon PostgreSQL (described in DEPLOY.md)	Free hosting; not run on Vercel from the machine used for this report

3.1 Running the project

The project is a folder named `quizbank`. In a terminal opened in that folder:

```
python -m venv venv
venv\Scripts\activate              # Windows (Mac/Linux: source
venv/bin/activate)
pip install -r requirements.txt
python seed.py                      # sample users, 24 questions, 2 quizzes, 8
demo students (once)
uvicorn app.main:app --reload       # web page at http://127.0.0.1:8000, API
documentation at /docs
pytest                              # the 13 automated tests
```

Table 3.2: Accounts created by seed.py

Role	E-mail	Password
admin	admin@quiz.com	admin123
teacher	teacher@quiz.com	teacher123
student	student@quiz.com	student123
8 demo students	demo1@quiz.com ... demo8@quiz.com	demo1234

4. ARCHITECTURE DIAGRAM

The system has four layers (Figure 4.1). Clients, which can be the web page, the Swagger UI or any HTTP client, talk to the API only through HTTP and JSON. The API is organised as four routers, one for each group of resources: users, the question bank, quizzes and attempts, and learning (practice, revision and mastery). Two concerns cut across all of them: authentication and the role check, and the validation of input by Pydantic schemas. At the bottom, a small database module gives every router the same interface to SQLite or PostgreSQL.

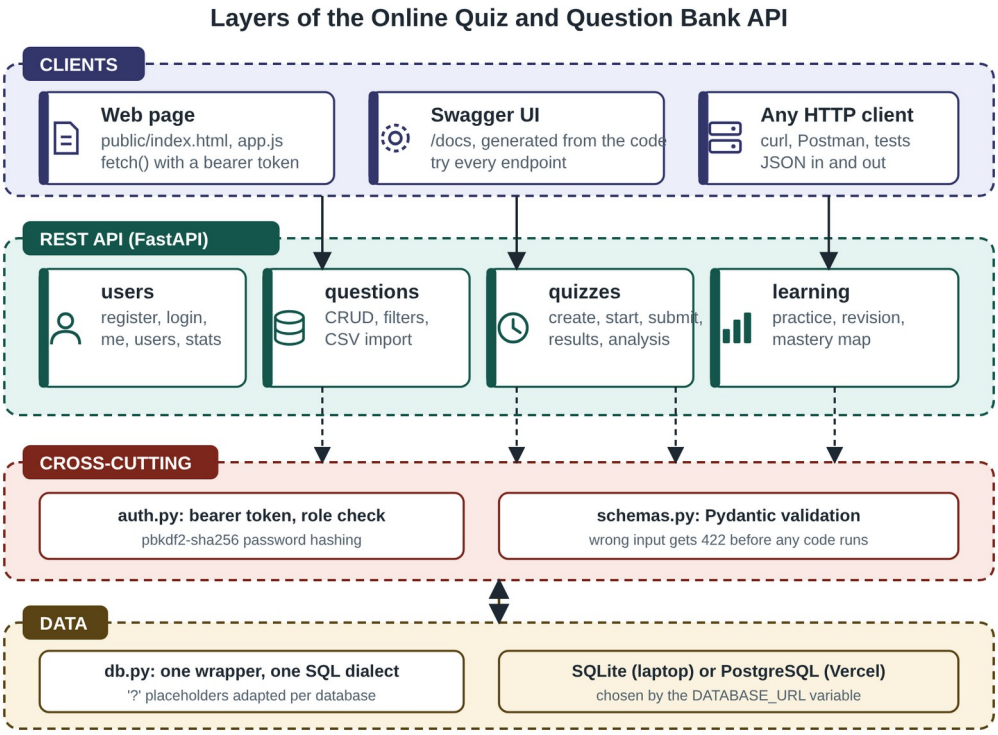


Figure 4.1: Layers of the Online Quiz and Question Bank API

Table 4.1: Module responsibilities

File	Lines	Responsibility
app/main.py	27	Creates the FastAPI application, mounts the four routers and the web page
app/routes_users.py	55	Register, login, me, user list, delete user, statistics
app/routes_questions.py	102	Question bank: CRUD, filters, paging, CSV import
app/routes_quizzes.py	286	Quizzes, attempts, marking, results, CSV export, item analysis, leaderboard
app/routes_learn.py	130	Adaptive practice, revision queue, mastery map
app/learning.py	42	Leitner box update and calibration score
app/auth.py	45	Password hashing, tokens, current-user and role dependencies
app/schemas.py	88	Pydantic models and validation rules
app/db.py	179	Schema, connection wrapper for SQLite and PostgreSQL
public/ (3 files)	416	The web page that uses the API
tests/test_api.py	237	13 automated tests
seed.py	115	Sample data

4.1 Resources and operations

The API has 29 operations on 24 addresses (Table 4.2): 6 for users, 7 for the question bank, 12 for quizzes and attempts and 4 for learning. The last column shows who may call each operation; the check is made by a dependency that reads the bearer token, finds the user and compares the role.

Table 4.2: Operations of the API

Method	Path	What it does	Allowed
POST	/api/register	Register	public
POST	/api/login	Login	public
GET	/api/me	Me	any user
GET	/api/users	List Users	admin
DELETE	/api/users/{user_id}	Delete User	admin
GET	/api/stats	Stats	admin, teacher
GET	/api/subjects	Subjects	any user
GET	/api/questions	List Questions	admin, teacher
POST	/api/questions	List Questions	admin, teacher
POST	/api/questions/import	Import Questions	admin, teacher
GET	/api/questions/{question_id}	Get Question	admin, teacher
PUT	/api/questions/{question_id}	Get Question	admin, teacher
DELETE	/api/questions/{question_id}	Get Question	admin, teacher
POST	/api/quizzes	Create Quiz	admin, teacher
GET	/api/quizzes	Create Quiz	any user
POST	/api/quizzes/random	Random Quiz	admin, teacher
GET	/api/quizzes/{quiz_id}	Get Quiz	admin, teacher
DELETE	/api/quizzes/{quiz_id}	Get Quiz	admin, teacher
POST	/api/quizzes/{quiz_id}/start	Start Attempt	student
POST	/api/attempts/{attempt_id}/submit	Submit Attempt	student
GET	/api/attempts/mine	My Attempts	student
GET	/api/quizzes/{quiz_id}/results	Quiz Results	admin, teacher
GET	/api/quizzes/{quiz_id}/results.csv	Results Csv	admin, teacher
GET	/api/quizzes/{quiz_id}/analysis	Item Analysis	admin, teacher
GET	/api/quizzes/{quiz_id}/leaderboard	Leaderboard	any user
GET	/api/practice/next	Practice Next	student

Method	Path	What it does	Allowed
POST	`/api/practice/answer`	Practice Answer	student
GET	`/api/revision/due`	Revision Due	student
GET	`/api/me/mastery`	Mastery	student

4.2 Database

Nine tables hold the data (Figure 4.2). `users` and `tokens` hold accounts and sessions. `questions` is the question bank, `quizzes` the quizzes and `quiz_questions` the many-to-many link between them. `attempts` records every attempt of a student with its start time, deadline, status, score and calibration, and `answers` the choices and confidence levels given in it. `practice_log` records the adaptive practice answers and `revision` holds the Leitner box and due date of each question for each student. Deleting a user, a quiz or an attempt deletes the rows that depend on it, and a question that belongs to a quiz cannot be deleted, so that the history of attempts stays intact.

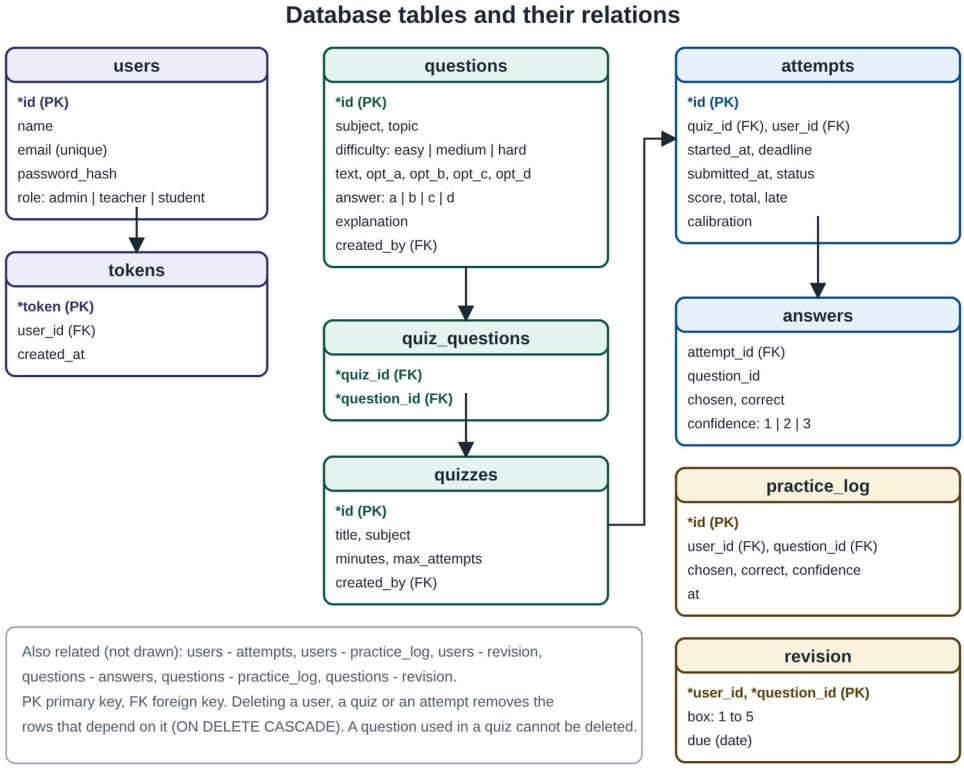


Figure 4.2: Database tables and their relations

5. METHODOLOGY

5.1 Overall approach

The API was designed resource first: the nouns (users, questions, quizzes, attempts) were decided before the code, then the operations on each were mapped to HTTP methods, and the rules that the server must enforce were written down and turned into tests. FastAPI generates the OpenAPI document from the code, so the documentation cannot drift from the behaviour. The learning features (calibration, revision, adaptive practice, item analysis) are kept in small functions with simple formulas, so each can be checked by hand and by a simulation.

5.2 Status codes

Table 5.1: Status codes used and when

Code	Meaning	Example in this API
200 OK	request succeeded	listing questions, login, results
201 Created	a resource was created	register, add a question, create a quiz, start or submit an attempt
204 No Content	success with no body	delete a question, quiz or user
400 Bad Request	valid JSON but impossible request	asking for 40 random questions when only 8 match; deleting yourself; a CSV without the required columns
401 Unauthorized	no or invalid token, or wrong password	calling /api/me without a token; wrong password at login
403 Forbidden	known user, not allowed	a student adding a question; a teacher listing users
404 Not Found	no such resource	results of a quiz that does not exist
409 Conflict	clashes with the current state	registering an e-mail twice; submitting an attempt twice; no attempts left
422 Unprocessable Entity	input fails validation	difficulty 'extreme'; role 'admin' at registration; a name of one letter

5.3 Authentication and roles

A password is never stored. At registration the server draws a random 16-byte salt and stores the salt together with the PBKDF2-HMAC-SHA256 digest of the password computed with 100,000 iterations [4]; at login the digest is recomputed and compared with `hmac.compare_digest`, which takes the same time whatever the input. A successful login creates a random 24-byte token (48 hexadecimal characters) from the `secrets` module, stores it with the user and returns it. The client sends it in the header `Authorization: Bearer <token>` [3]. A dependency, `current_user`, reads the header, looks up the token and returns the user or answers 401. A second dependency, `need_role(...)`, wraps it and answers 403 when the role is not in the list. A route is

protected simply by naming the dependency in its signature. The role 'admin' cannot be chosen at registration (422); the first administrator is created by the seed script.

Table 5.2: Who may do what

Operation	Admin	Teacher	Student
Register, login, see own profile	yes	yes	yes
Manage users, delete users	yes	no	no
See statistics	yes	yes	no
Create, edit, delete questions and quizzes; import CSV	yes	yes	no
See results, item analysis, CSV export	yes	yes	no
List quizzes, see leaderboard	yes	yes	yes
Start and submit attempts; practice; revision; mastery	no	no	yes

5.4 Validation

Every request body is a Pydantic model. The rules are declared next to the fields, for example `name` 2 to 60 characters, `email` matching a pattern, `password` at least 6 characters, `difficulty` one of easy, medium or hard, `answer` one of a, b, c or d, `minutes` from 1 to 180, `count` from 1 to 50 and `confidence` from 1 to 3. If any rule fails, FastAPI answers 422 with a list of the problems and the route function never runs. Query parameters such as `page` and `size` are validated in the same way (`size` at most 100). All SQL uses placeholders, never string concatenation, so a hostile value cannot change the query.

5.5 The question bank

A question has a subject, a topic, a difficulty, a text, four options, the correct option and an explanation that is shown after answering. The list operation accepts the filters `subject`, `topic`, `difficulty` and `search` (a case-insensitive part of the text) and returns a page with the total count. The import operation takes the text of a CSV file, checks that the required columns exist, validates each line with the same model as a single question, adds the good lines and returns the number added and the list of bad lines with the reason, so that one mistake does not lose a whole file. A question that belongs to a quiz cannot be deleted (409), because the results of earlier attempts refer to it.

5.6 Quizzes and attempts

A quiz is made from chosen question ids or, with the random operation, from the questions that match a subject, topic and level; the server asks the database for random rows and answers 400 if fewer exist than were asked for. Figure 5.1 shows the life of an attempt.

When a student starts an attempt the server checks that attempts remain, stores the start time and the **deadline** (start plus the minutes of the quiz) and returns the paper without the correct answers. If an open attempt already exists it is resumed, with the time that is really left. The order of the questions and of the options is shuffled with a random generator seeded from the attempt id, so each student has a different paper, and reloading the page shows the same one.

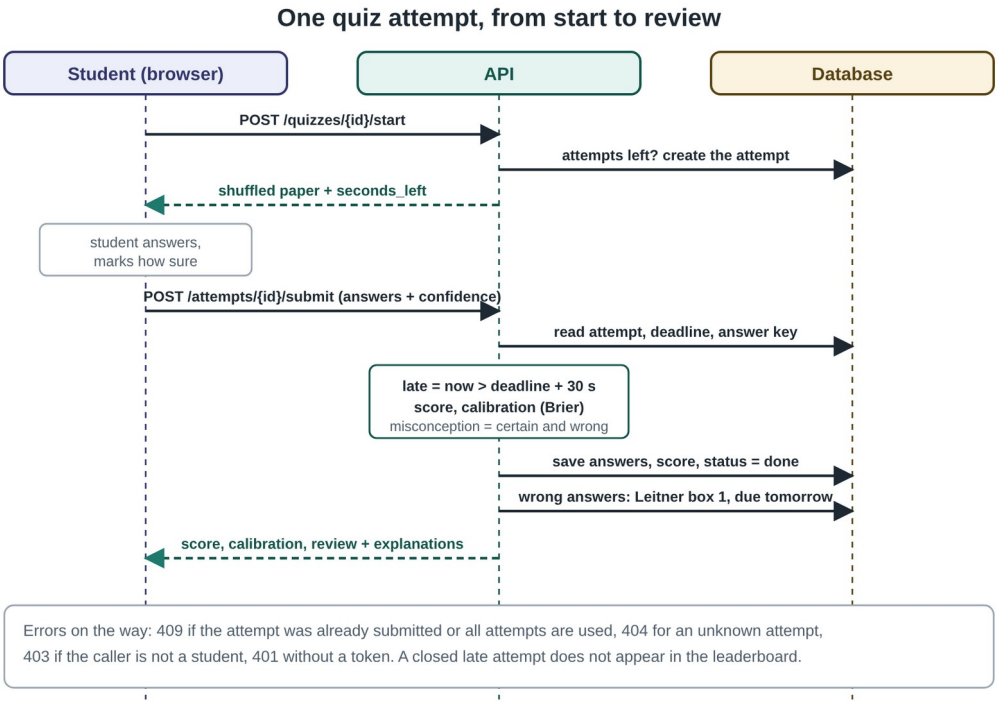


Figure 5.1: The life of one quiz attempt

When the answers arrive, the server compares the time with the deadline plus a grace period of 30 seconds for network delay; a later submission is still marked but flagged `late`, and late attempts are left out of the average, the leaderboard and the item analysis. An attempt that was never submitted before its deadline is closed with a score of zero. The timer shown in the browser is only a display: it starts from `seconds_left`, which the server computes, and the server does not trust the browser's clock. An attempt can be submitted once (a second submission is 409), and the number of attempts is limited per quiz (409 when used up).

5.7 Confidence calibration

With each answer the student gives a confidence level: 1 guessing, 2 fairly sure, 3 certain. Each level is treated as a claim about the chance of being right: 0.33, 0.66 and 0.95. For n answered questions, with outcome o (1 right, 0 wrong) and claim c, the Brier score [5] is the mean of $(c - o)^2$, and the **calibration score** is 100 × (1 − Brier score), so 100 is perfect.

A student who is sure only when right scores high; a student who is certain and wrong is penalised heavily; a student who honestly guesses is hardly penalised. Worked example: four answers with marks (right, certain), (right, certain), (wrong, certain), (right, fairly sure) give squared errors $0.05^2 + 0.05^2 + 0.95^2 + 0.34^2 = 0.0025 + 0.0025 + 0.9025 + 0.1156 = 1.0231$, a Brier score of 0.2558 and a calibration score of 74. In addition, an answer that is wrong and given with confidence 3 is flagged as a **misconception** in the review.

5.8 Spaced revision

Wrong answers are scheduled for revision with the Leitner system [6]. A question has a box from 1 to 5, and the number of days until it is due again is 1, 2, 4, 8 and 16 for boxes 1 to 5. A wrong answer puts the question in box 1, due tomorrow, or moves it back there if it was already in the queue. A right answer to a question that is in the queue moves it up one box, and a right answer in box 5 removes it from the queue, so a question is learned after five right answers in a row on the due days. If every review were right, the question would be reviewed on days 1, 3, 7, 15, 31 after the first mistake. Questions due today are served by `/api/revision/due`; both quizzes and practice feed the queue.

5.9 Adaptive practice

In practice mode the server serves one question at a time. If the student names a topic the question comes from it; otherwise the server chooses a topic: with probability one half a topic never tried, otherwise the topic with the lowest accuracy among those with at least two answers (accuracy is computed from practice answers and finished quizzes together). The **difficulty** follows the last six practice answers of that subject: the base level is the average level of those questions (easy 1, medium 2, hard 3) rounded; if the accuracy is at least 80% the level goes up by one, if it is at most 40% it goes down by one, and with fewer than three answers the level is easy. The question is chosen at random from the unseen ones of that level (the eight most recent are skipped when possible), and the reply says why this question was chosen. The mastery map classifies each topic with at least three answers as strong (75% or more), learning (50% or more) or weak.

5.10 Item analysis

For a finished quiz with at least one on-time attempt the teacher can ask for item analysis [8][9]. The **difficulty index** p of a question is the share of students who answered it correctly. The **discrimination index** is $D = p(\text{upper group}) - p(\text{lower group})$, where the

groups are the 27% of students with the highest and the 27% with the lowest total score [8]; it needs at least four students. A question gets advice when it is very easy (*p* above 0.9), very hard (*p* below 0.3), poorly discriminating (*D* below 0.2), or when its measured difficulty disagrees with its label (measured easy at *p* of 0.7 or more, hard below 0.4, otherwise medium). The report also names the wrong option chosen most often, which points to the misconception behind a question. The **leaderboard** takes each student's best on-time attempt, ordered by score and then by the time taken.

5.11 One code base, two databases

All queries are written once with '?' placeholders. When the environment variable `DATABASE_URL` is not set the wrapper opens a SQLite file; when it is set, it opens a PostgreSQL connection, replaces '?' by '%s' and adds `RETURNING id` to inserts, and the table definitions use the right auto-increment type. A serverless host such as Vercel has no disk that survives between requests, which is why a hosted database is needed there. A connection is opened per request, committed only if the request succeeded and rolled back otherwise.

5.12 Testing methodology

The 13 tests use FastAPI's TestClient, so they send real HTTP requests to the application and check the status codes and the JSON, with a throw-away database. They cover registration and roles, question validation and filters, CSV import, protection of used questions, random quizzes, a full attempt with confidence and revision, attempt limits, the server timer (by moving the deadline into the past), item analysis and leaderboard, the calibration formula, adaptive practice and revision spacing, the mastery map and the serving of the web page and documentation. The experiments of Chapter 6 add simulated students: they use the real API and the real functions, with answers drawn from a random generator with a fixed seed.

6. RESULTS

6.1 Test results

All 13 automated tests pass in about three seconds (Figure 6.1). Table 6.1 says what each one checks.

```
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/user/CleanEnergy/quizbank
plugins: anyio-4.15.1
collecting ... collected 13 items
tests/test_api.py::test_register_login_roles PASSED                      [  7%]
tests/test_api.py::test_question_crud_validation_and_filters PASSED      [ 15%]
tests/test_api.py::test_csv_import_reports_bad_lines PASSED              [ 23%]
tests/test_api.py::test_used_question_cannot_be_deleted PASSED           [ 30%]
tests/test_api.py::test_random_quiz PASSED                               [ 38%]
tests/test_api.py::test_full_quiz_flow_with_confidence_and_revision PASSED [ 46%]
tests/test_api.py::test_attempt_limit_and_roles PASSED                   [ 53%]
tests/test_api.py::test_timer_is_enforced_by_the_server PASSED           [ 61%]
tests/test_api.py::test_item_analysis_and_leaderboard PASSED             [ 69%]
tests/test_api.py::test_calibration_score PASSED                         [ 76%]
tests/test_api.py::test_practice_adapts_and_revision_spaces_questions PASSED [ 84%]
tests/test_api.py::test_mastery_points_to_weak_topics PASSED             [ 92%]
tests/test_api.py::test_home_page_and_docs_are_served PASSED             [100%]
============================== 13 passed in 2.92s ==============================
```

Figure 6.1: Output of pytest -v

Table 6.1: Automated tests

Test	What is checked
register_login_roles	201 on registration, 409 for a second registration, 401 for a wrong password and for no token, 422 when asking to be admin, 403 for a teacher listing users
question_crud_validation_and_filters	a student cannot add; teacher adds, reads, updates; invalid values give 422; filters and paging
csv_import_reports_bad_lines	good lines are added, bad lines are reported with the reason
used_question_cannot_be_deleted	409 for a question that belongs to a quiz
random_quiz	random quiz of the requested size; 400 when too few questions exist
full_quiz_flow_with_confidence_and_revision	start, shuffled paper, submit, score, calibration, misconception flag, entry in the revision queue
attempt_limit_and_roles	409 when attempts are used up; roles on the attempt routes
timer_is_enforced_by_the_server	an attempt past its deadline is marked late and kept out of the ranking
item_analysis_and_leaderboard	difficulty and discrimination index, flags, ranking
calibration_score	the formula on known inputs
practice_adapts_and_revision_spaces_questions	difficulty follows results; boxes and due dates
mastery_points_to_weak_topics	mastery map and focus list
home_page_and_docs_are_served	the web page, /docs and /openapi.json respond

6.2 The API in Swagger UI

FastAPI generates the OpenAPI document of the API and the interactive page of Figure 6.2 at `/docs`. The 29 operations are grouped as users, question bank, quizzes and learning, each with its method, path and summary, and each can be tried from the page with a token. The same document can be imported into Postman or used to generate a client.

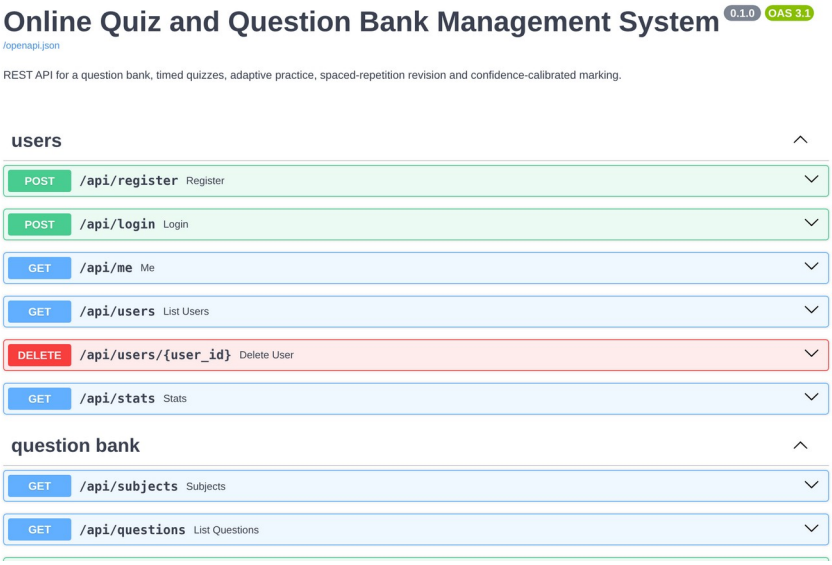


Figure 6.2: Swagger UI generated from the code (/docs)

6.3 Requests and responses

Table 6.2 lists real requests made to the API by the benchmark script and the answers received. The same operation gives different status codes depending on who calls it and with what: for example POST /api/questions returns 201 for a teacher, 403 for a student and 422 for wrong values.

Table 6.2: Requests made and status codes returned

Method	Path	Situation	Status
POST	`/api/register`	Register a student	201
POST	`/api/register`	Register the same e-mail again	409
POST	`/api/register`	Try to register as admin	422
POST	`/api/login`	Login with a wrong password	401
POST	`/api/login`	Login	200
GET	`/api/me`	Call a protected route without a token	401
POST	`/api/questions`	A student tries to add a question	403
POST	`/api/questions`	A teacher adds a question	201
POST	`/api/questions`	Add a question with wrong values	422
GET	`/api/questions?subject=REST API&difficulty=e`	List questions with filters	200
POST	`/api/quizzes/random`	Ask for more questions than exist	400
POST	`/api/quizzes/random`	Create a random quiz	201
POST	`/api/quizzes/1/start`	Start an attempt	201
POST	`/api/attempts/1/submit`	Submit the answers with confidence levels	201
POST	`/api/attempts/1/submit`	Submit the same attempt twice	409
GET	`/api/revision/due`	Questions waiting for revision	200
GET	`/api/quizzes/1/results`	Teacher: results of a quiz	200
GET	`/api/users`	A teacher asks for the user list	403
GET	`/api/quizzes/9999/results`	Results of a quiz that does not exist	404

Two of the exchanges in full. First, a student registers and logs in:

```
POST /api/register   {"name": "Asha", "email": "asha@example.com", "password":
"secret1", "role": "student"}
201 {"id": 1, "name": "Asha", "email": "asha@example.com", "role": "student"}
```

```
POST /api/login      {"email": "asha@example.com", "password": "secret1"}
200 {"token": "ff418bb53376...", "name": "Asha", "role": "student"}
```

Second, the student submits a quiz of four questions, giving a confidence level with each answer. The server returns the score, the calibration score and a review in which an answer that was wrong and certain is marked as a misconception:

```
POST /api/attempts/1/submit
Authorization: Bearer <token>
{"answers": {"<question id>": {"choice": "b", "confidence": 3}, ...}}

201 {"score": 3, "total": 4, "percent": 75.0, "late": false, "calibration":
72, "review": [{"correct": true, "misconception": false}, {"correct": false,
"misconception": true}, {"correct": true, "misconception": false}, {"correct":
true, "misconception": false}]}
```

6.4 Screens of the web page

Figures 6.3 to 6.13 are screenshots of the web page that uses the API, taken from the running system with the sample data of `seed.py`. The page is deliberately plain, since the API is the subject of the project. The top bar shows who is logged in, and the tabs depend on the role.

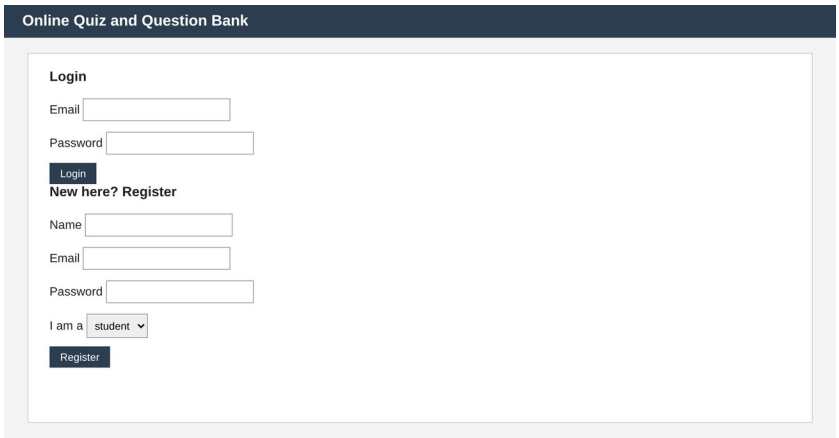


Figure 6.3: Login and registration (a registrant may choose only student or teacher)

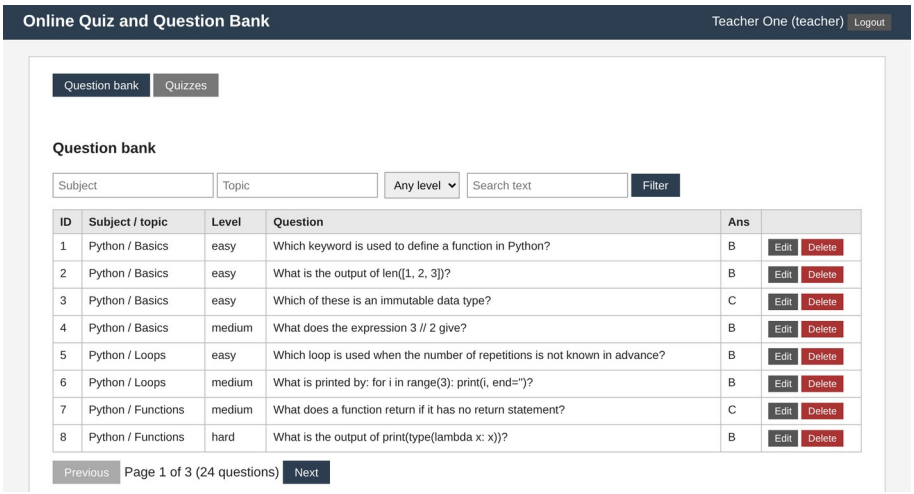


Figure 6.4: Teacher: question bank, 24 questions in pages of 10

In Figure 6.4 the teacher sees the question bank with filters for subject, topic, level and text, the Edit and Delete buttons, and the paging controls that call `GET /api/questions?page=...`. Filtering by subject Python and level hard returns 2 of the 24 questions (Figure 6.5, which is the same request with the query parameters `subject=Python&difficulty=hard`).

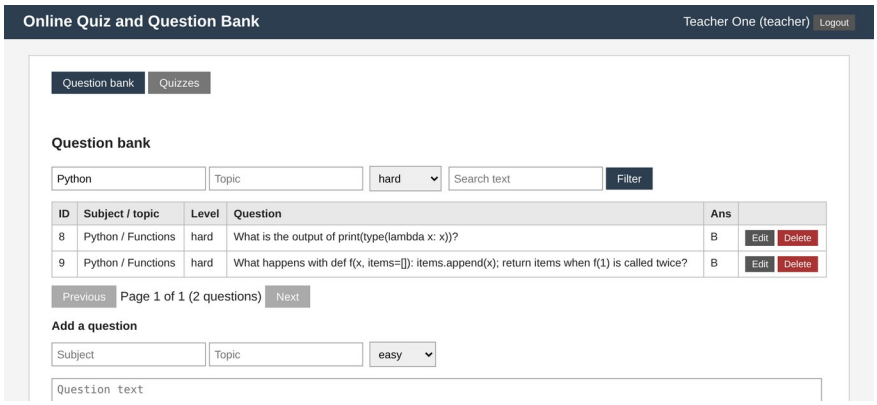


Figure 6.5: Teacher: the filter applied (Python, hard)

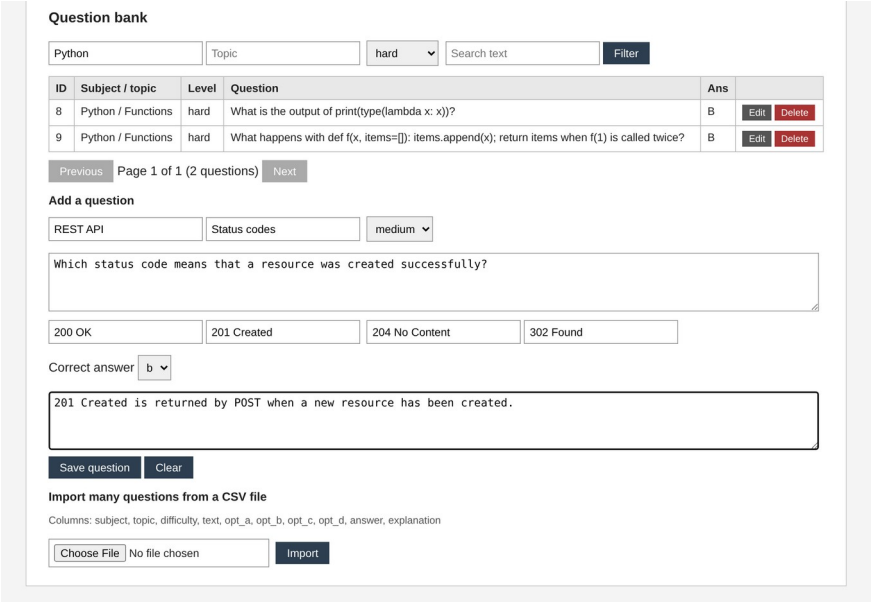


Figure 6.6: Teacher: adding a question with its explanation

Figure 6.6 shows the form for a new question: subject, topic, level, text, four options, the correct option and the explanation that the student will see after answering. Saving calls `POST /api/questions`; values outside the allowed set are returned as 422 and shown in the message line.

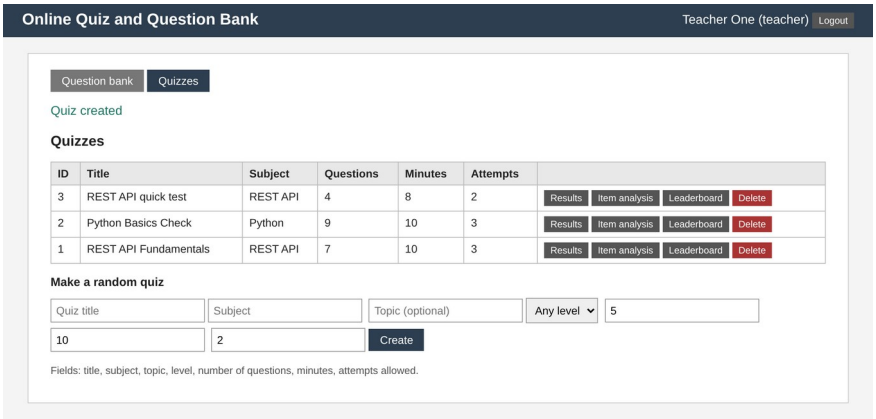


Figure 6.7: Teacher: quiz list and the form that creates a random quiz

In Figure 6.7 the teacher has just created a quiz of four random REST API questions with eight minutes and two attempts (`POST /api/quizzes/random`). Each quiz has buttons for results, item analysis, leaderboard and delete.

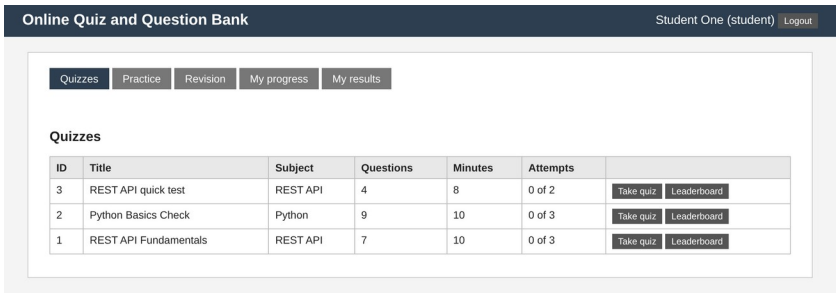


Figure 6.8: Student: quizzes with the attempts used

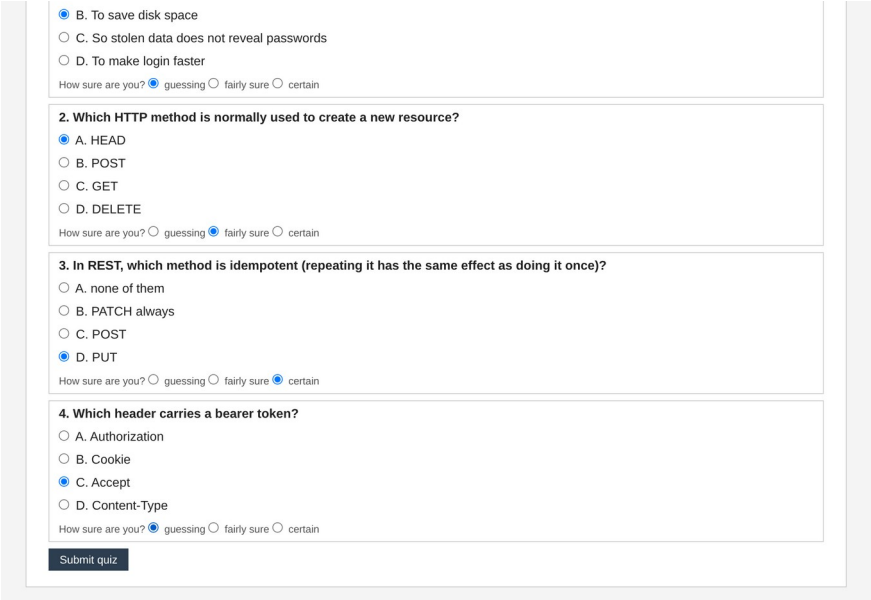


Figure 6.9: Student: a quiz in progress, with a confidence level for every question

Figure 6.9 shows a quiz in progress. The student sees only the question text and the options, in an order that is different for every attempt, and answers each question with a confidence level: guessing, fairly sure or certain. The countdown comes from the server's `seconds_left`.

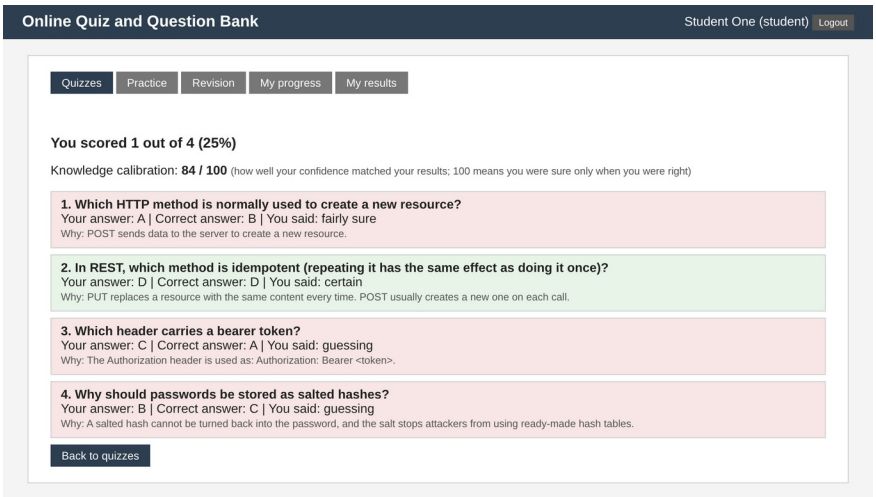


Figure 6.10: Student: score, calibration and review with explanations

After submission (Figure 6.10) the student sees the score (here 1 of 4), the knowledge calibration (84 of 100), and for each question the correct answer, the confidence given and the explanation. Questions answered wrongly were added to the revision list.

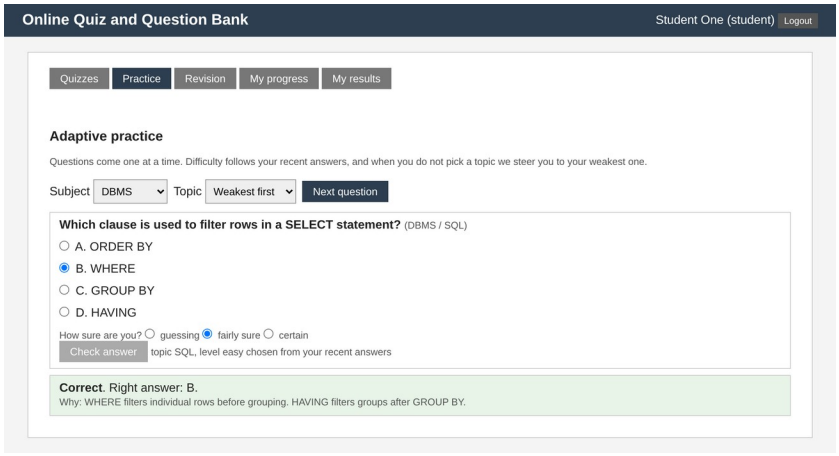


Figure 6.11: Student: adaptive practice, one question at a time

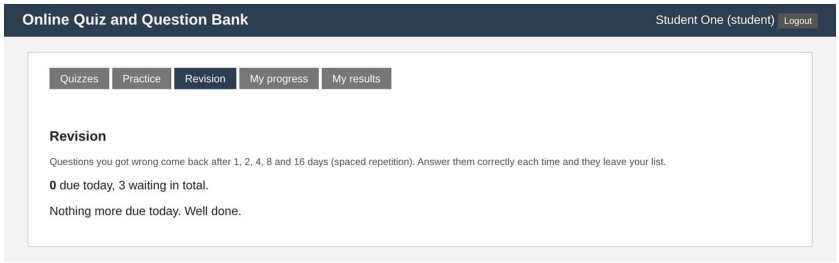


Figure 6.12: Student: revision list (spaced repetition)

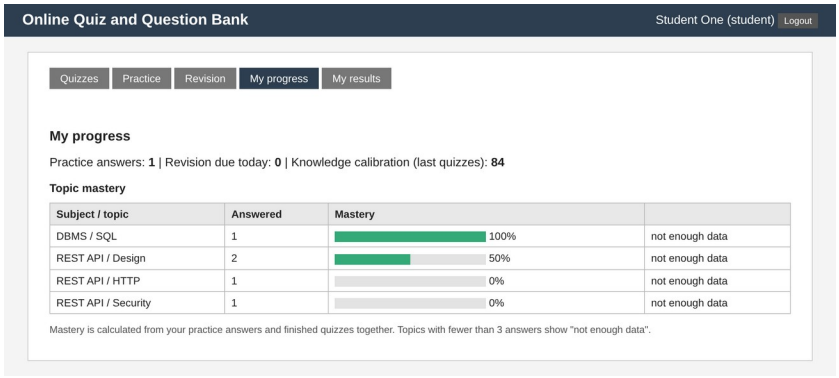


Figure 6.13: Student: topic mastery map

Figure 6.11 shows adaptive practice with the correct answer and the explanation after checking. Figure 6.12 shows the revision page: 3 questions are waiting, none is due today, because a question answered wrongly is due tomorrow. Figure 6.13 shows the mastery map computed from practice and quizzes together; topics with fewer than three answers are marked as not having enough data.

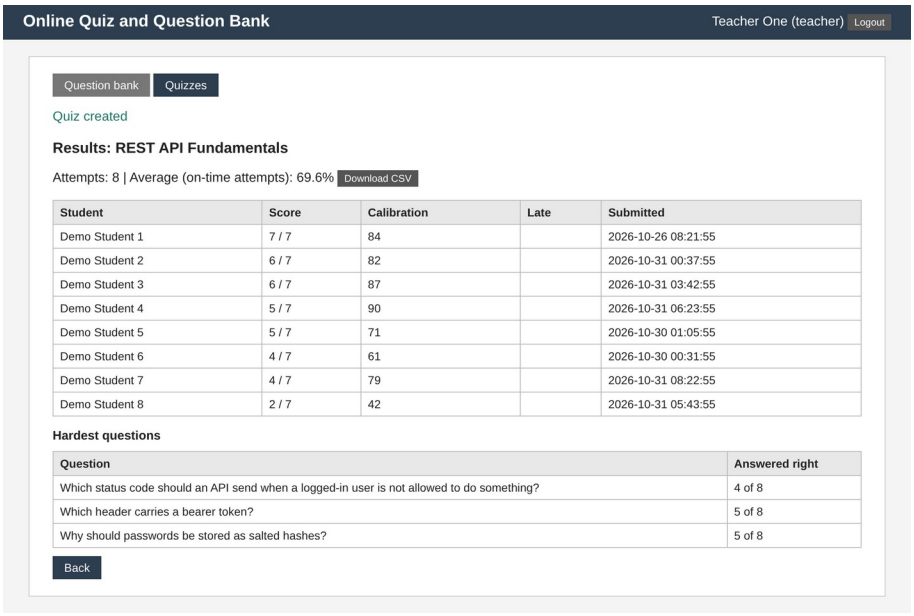


Figure 6.14: Teacher: results of a quiz with calibration and the hardest questions

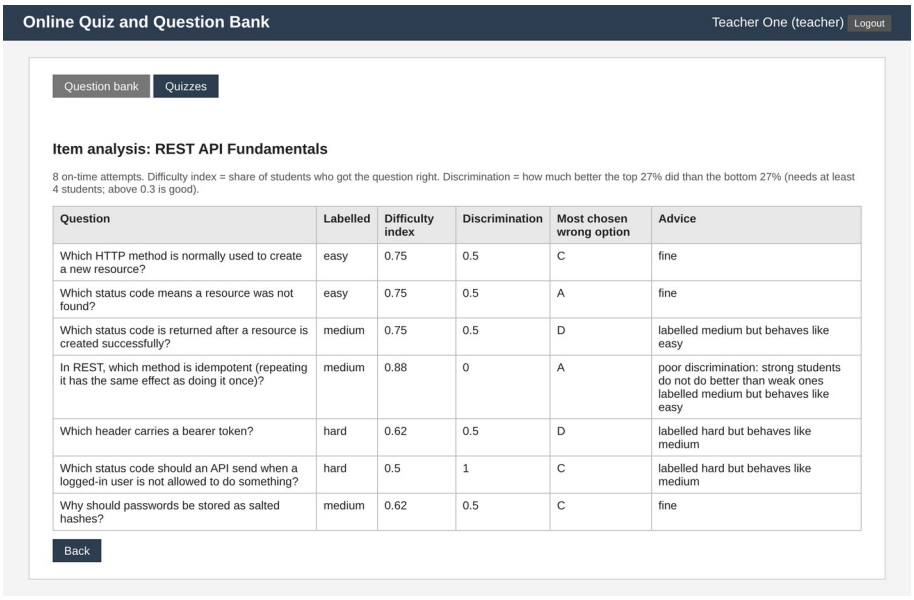


Figure 6.15: Teacher: item analysis

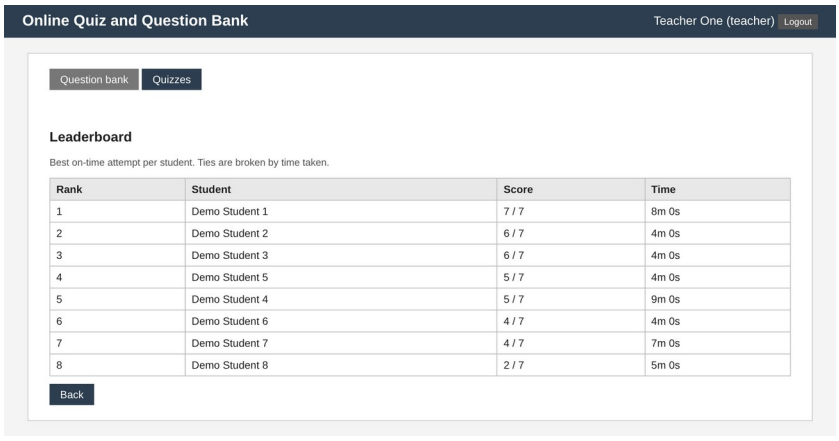


Figure 6.16: Leaderboard

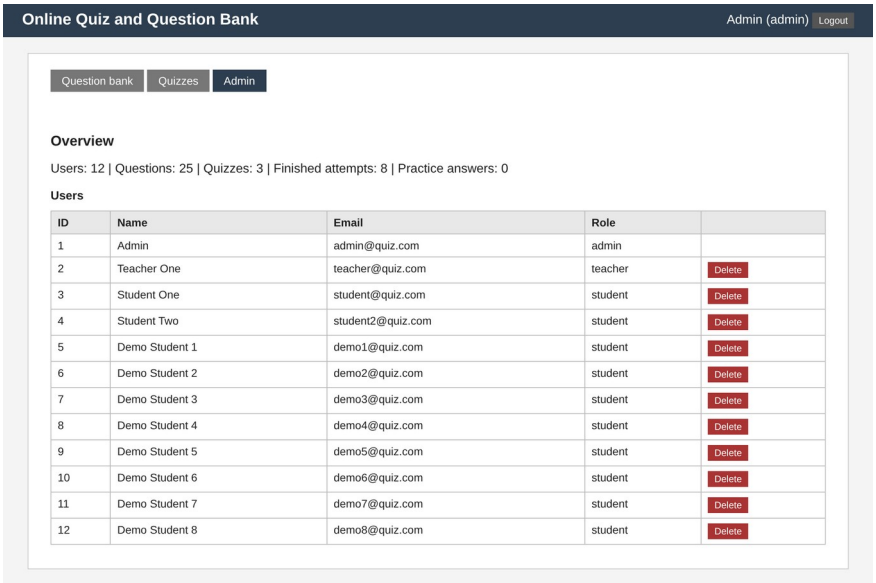


Figure 6.17: Admin: statistics and user management

Figure 6.14 shows the results of the quiz *REST API Fundamentals* for the 8 demonstration students created by the seed script: the average of 69.6%, each student's score and calibration, and the three hardest questions. Figure 6.15 is the item analysis of the same quiz; it is discussed in Section 6.8. Figure 6.16 is the leaderboard, and Figure 6.17 the administrator's page with the statistics (12 users, 24 questions, 2 quizzes, 8 finished attempts) and the list of users with a Delete button. The demonstration students and their attempts are generated by the seed script; they are not real students.

6.5 Does the calibration score separate honest from over-confident students?

To test the calibration score, four kinds of student were simulated. Each answers 10 questions, for each of which the chance of being right is drawn between 0.2 and 1.0, in 3,000 attempts per kind. The *calibrated* student knows how sure to be: certain when the chance is 0.85 or more, fairly sure from 0.5 to 0.85, guessing below. The *over-confident* student is always certain, the *under-confident* one always guessing, and the *random* one chooses a level at random. The scores come from the application's own function.

Table 6.3: Calibration score of four simulated kinds of student (3,000 attempts each)

Student	Mean accuracy	Mean calibration score	Standard deviation
calibrated	60.3%	80.4	5.8
overconfident	59.7%	63.8	14.1
underconfident	59.6%	68.9	5.4
random	60.0%	69.2	9.4

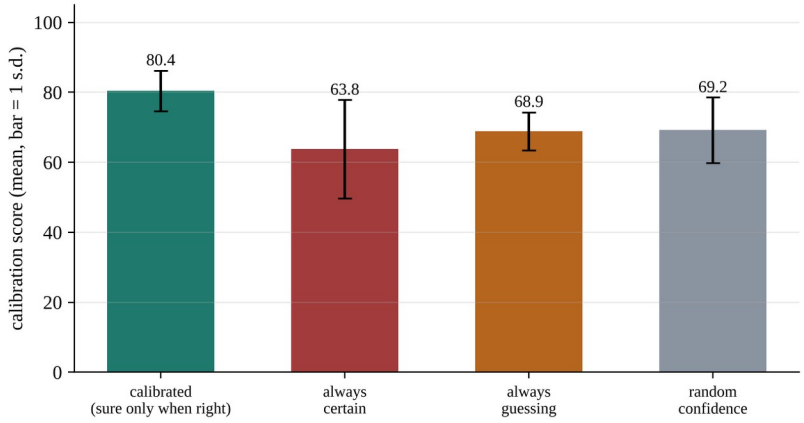


Figure 6.18: Calibration score by kind of student

All four kinds are right in about 60% of the questions, so the ordinary score cannot tell them apart. The calibration score does: the calibrated student averages 80, the always-certain student 64 with a much wider spread (14 against 6), because one confident mistake costs 0.9 and the cost is not averaged away in a short quiz. The always-guessing and the random student both score about 69. This is a statement about the formula and a simulation, not about real students: it shows that the measure has the property that a teacher would want, that it rewards knowing what you know.

6.6 Spaced revision

The revision queue was exercised through the real `update_revision` function with a simulated learner who answers correctly with a fixed probability p at every review, 400 learners for each value of p. A learner starts with a wrong answer and is reviewed on the due days until the question leaves the queue (five right answers in a row). With perfect recall the question leaves after 31 days and 5 reviews, as the schedule 1, 2, 4, 8, 16 says (reviews on days 1, 3, 7, 15, 31). Table 6.4 and Figure 6.19 show what happens when recall is not perfect.

Table 6.4: Reviews and days needed to learn one question

Chance of a right answer at a review	Mean reviews	Mean days	Median days	Most reviews
0.5	62.0	159	116	338
0.7	16.6	66	48	83
0.9	7.3	39	31	25
0.97	5.4	32	31	13

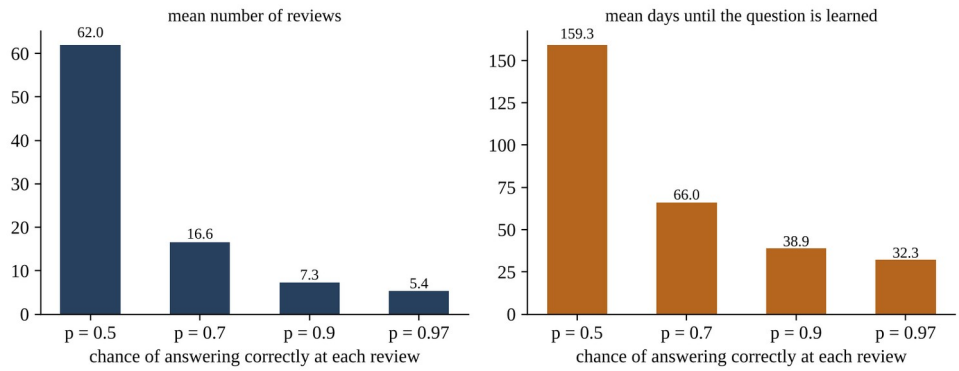


Figure 6.19: Effort needed to learn a question for different recall probabilities

A learner who recalls the answer 9 times out of 10 needs about 7 reviews over 39 days; one who recalls it half of the time needs 62 reviews over 159 days, because each failure sends the question back to box 1. The scheme therefore spends the effort where it is needed: easy questions leave the queue after five short reviews, hard ones keep returning at short intervals until they are learned. A weakness of the simple rule is also visible: a single failure in box 5 throws away four reviews of progress.

6.7 Adaptive practice

Three simulated students used the practice mode through the HTTP API, 12 students of each kind, 30 questions each, on a subject with 30 questions at each of the three levels. The weak student answers correctly 80%, 45% and 15% of the easy, medium and hard questions, the average student 92%, 70% and 40%, the strong student 98%, 90% and 75%. Figure 6.20 shows the levels of the questions served after the first ten, once the mechanism has adapted.

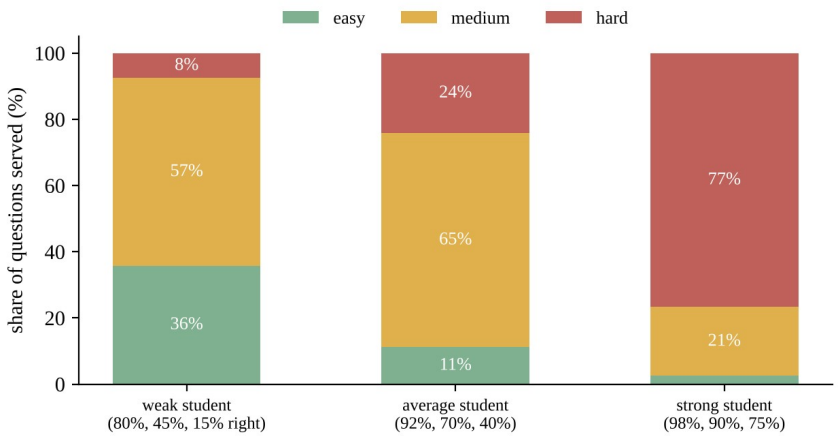


Figure 6.20: Difficulty of the practice questions served, by kind of student (questions 11 to 30)

The strong student received 77% hard questions, the average student 24% and the weak student 8%; the weak student worked mostly at medium level (57%) and was given 36%

easy questions. The difficulty follows the ability of the student, as intended. The weak student is not held at the easiest level, because a score of about 45% on medium questions is above the 40% threshold for stepping down. The experiment checks the behaviour of the rule; whether the questions really help students to learn would need a study with real students.

6.8 Item analysis of the demonstration quiz

Table 6.5 is the item analysis of *REST API Fundamentals* (8 on-time attempts). The measured difficulty disagrees with the label for three questions, which the report marks (for example a question labelled hard that 62% of the students answered correctly). One question has a discrimination index of 0: strong and weak students did equally well, so it does not help to tell them apart. These are exactly the questions that a teacher should reword or relabel. The attempts of the demonstration students are generated, so the figures illustrate the report and say nothing about real students.

Table 6.5: Item analysis of REST API Fundamentals

Question	Label	Difficulty index	Discrimination	Wrong option most chosen	Advice
Which HTTP method is normally used to create a new resourc...	easy	0.75	0.5	C	fine
Which status code means a resource was not found?	easy	0.75	0.5	A	fine
Which status code is returned after a resource is created ...	medium	0.75	0.5	D	labelled medium but behaves like easy
In REST, which method is idempotent (repeating it has the ...	medium	0.88	0.0	A	poor discrimination: strong students do not do better than weak ones; labelled medium but behaves like easy
Which header carries a bearer token?	hard	0.62	0.5	D	labelled hard but behaves like medium
Which status code should an API send when a logged-in user...	hard	0.5	1.0	C	labelled hard but behaves like medium
Why should passwords be stored as salted hashes?	medium	0.62	0.5	C	fine

6.9 Security checks

The behaviour that protects the system is part of the tests and of Table 6.2: a student cannot add questions (403), a teacher cannot list users (403), a request without a token is refused (401), a wrong password gives the same 401 as an unknown e-mail, an attempt cannot be submitted twice (409), and the role 'admin' cannot be requested at registration (422). The paper sent to a student contains no answers, and the review that does contain them is returned only after the attempt is closed. A late submission is accepted but marked and cannot enter the leaderboard. Weaknesses that remain are listed in Section 7.1.

6.10 Performance

Table 6.6: Time per request (FastAPI TestClient on one laptop-class machine)

Request	Median (ms)	95th percentile (ms)
POST /api/login	36.8	54.3
GET /api/questions	4.8	6.2
GET /api/quizzes	4.3	5.5
GET /api/practice/next	5.2	6.7
start + submit a 20-question quiz	14.8	15.9
GET /api/quizzes/{id}/analysis	5.6	6.5

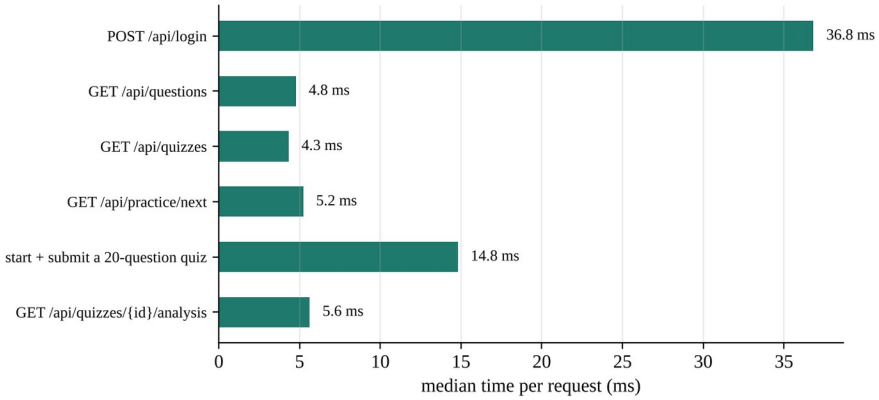


Figure 6.21: Median time per request

Most requests take about 5 ms. The slowest is the login (37 ms), and it is slow on purpose: PBKDF2 with 100,000 iterations is what makes guessing passwords expensive. Starting and submitting a 20-question quiz together takes about 15 ms. These times were measured in-process without a network, so they show the cost of the application code only; a deployed server adds the network and, on a serverless host, the start of the function.

6.11 Discussion

The results show that the API does what it was designed to do and that the extra features behave as their formulas say. The tests confirm the REST behaviour, in particular the status codes and the role checks. The simulation shows that the calibration score reveals something that the ordinary score cannot, that the revision queue concentrates effort on the questions that are hard for the learner, and that adaptive practice gives a strong student hard questions and a weak one mostly medium ones. The item analysis gives teachers a concrete list of questions to improve.

The evidence also has limits that should be stated. The students in the experiments are simulated with probabilities chosen by the authors, and the demonstration quiz data is generated, so no claim is made about learning outcomes with real students. The claimed

probabilities of the three confidence levels (0.33, 0.66, 0.95) are a modelling choice. The discrimination index needs at least four students and is unreliable for small groups. The deployment on Vercel with PostgreSQL was described and the database layer was tested against PostgreSQL 16, but the project was not run on Vercel itself for this report.

7. CONCLUSION

This mini project built the REST API of an online quiz and question-bank system with 29 operations, three roles, a validated and documented interface and a database layer that runs on SQLite and PostgreSQL. The server controls what must not be left to the browser: authentication, roles, the timer, the number of attempts, the shuffling of the paper and the exclusion of late attempts. On top of marking, the system measures how well a student's confidence matches the result, brings wrong answers back at growing intervals, chooses practice questions by weakest topic and recent performance, and gives teachers the difficulty and discrimination of every question. The 13 automated tests pass, and the simulations confirm the intended behaviour of the learning features.

Three results stand out. The ordinary score cannot separate a student who knows what they know from one who is always certain, but the calibration score does (80 against 64 at the same accuracy). The Leitner queue needs only 5 reviews for a question that is remembered and about 62 for one that is not. And the practice mode serves 77% hard questions to a strong student and 8% to a weak one.

7.1 Limitations

- Only single-answer multiple-choice questions are supported; there are no essay, numeric or multiple-answer questions.
- Tokens do not expire and there is no logout endpoint on the server; there is no e-mail verification, password reset or rate limiting on login.
- The grace period of 30 seconds and the confidence claims 0.33, 0.66 and 0.95 are design choices that were not tuned on real data.
- The experiments use simulated students and generated demonstration data; the effect on real learning has not been measured.
- The shuffle is derived from the attempt id and is predictable to someone who knows the code; proctoring is out of scope.
- The Vercel deployment was not exercised from the machine used for this report.

7.2 Future scope

- Add token expiry and logout, refresh tokens, and rate limiting on the login route.
- Support more question types: multiple answers, numeric answers, short text and images.

- Use the item analysis to retire or reword questions automatically, and fit an item-response model when enough attempts exist.
- Replace the fixed Leitner intervals by intervals fitted to each learner, and let the confidence level influence the interval.
- Add classes and assignments, so that a teacher can give a quiz to a group and compare groups.
- Add a real mobile or single-page client, and an export of results in a spreadsheet format.
- Run a pilot with a class and measure whether practice with calibration feedback improves later scores.

8. REFERENCE

[1] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, University of California, Irvine, 2000.

[2] R. Fielding and J. Reschke, Eds., "HTTP Semantics," RFC 9110, Internet Engineering Task Force, 2022.

[3] M. Jones and D. Hardt, "The OAuth 2.0 Authorization Framework: Bearer Token Usage," RFC 6750, Internet Engineering Task Force, 2012.

[4] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, Internet Engineering Task Force, 2000.

[5] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1–3, 1950.

[6] S. Leitner, So lernt man lernen: Angewandte Lernpsychologie. Freiburg: Herder, 1972.

[7] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted and D. Rohrer, "Distributed practice in verbal recall tasks: A review and quantitative synthesis," Psychological Bulletin, vol. 132, no. 3, pp. 354–380, 2006.

[8] T. L. Kelley, "The selection of upper and lower groups for the validation of test items," Journal of Educational Psychology, vol. 30, no. 1, pp. 17–24, 1939.

[9] L. Crocker and J. Algina, Introduction to Classical and Modern Test Theory. New York: Holt, Rinehart and Winston, 1986.

[10] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[11] OpenAPI Initiative, "OpenAPI Specification, version 3.1.0," 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[12] Pydantic, "Pydantic documentation." [Online]. Available: https://docs.pydantic.dev

[13] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan and D. T. Willingham, "Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology," Psychological Science in the Public Interest, vol. 14, no. 1, pp. 4–58, 2013.